

Day 52 – Hibernate Day 10

Many-to-Many Mapping (Bidirectional)

What is Many-to-Many Mapping?

Many-to-Many Mapping is used when multiple records from one table are connected with multiple records from another table.

In simple words:

One object can have many related objects.

Those related objects can also belong to many other objects.

Real-Life Example

Think about Students and Courses.

One Student can enroll in many Courses.

One Course can have many Students.

Both sides have multiple records connected with each other.

In Our Project

One Question can have multiple Answers.

One Answer can also be used for multiple Questions.

Example:

```
Question 1 → Answer 1, Answer 2, Answer 3
```

```
Question 2 → Answer 2
```

Here, Answer 2 belongs to both questions.

Relationship

Notice that Answer 2 is shared by both questions.

Question Entity (Owner Side)

```
package com.nit.Mapping3;

import java.util.List;
import jakarta.persistence.*;
```

```

import lombok.Data;

@Entity
@Table(name="que_value")
@Data
public class Question {

    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    @Column(name="que_id")
    private int id;

    private String questions;

    @ManyToMany(cascade = CascadeType.ALL)
    @JoinTable(
        name="question_answer",
        joinColumns=@JoinColumn(name="que_id"),
        inverseJoinColumns=@JoinColumn(name="ans_id")
    )
    private List<Answers> answers;
}

```

Explanation

- @Entity → Creates an entity.
- @Table(name="que_value") → Maps the class to the que_value table.
- id → Primary key.
- answers → Stores multiple Answer objects related to one Question.

This class is called the Owner Side because it creates the relationship table.

Answers Entity (Inverse Side)

```

package com.nit.Mapping3;

import java.util.List;
import jakarta.persistence.*;
import lombok.Data;

```

```

@Entity
@Table(name="ans_value")
@Data
public class Answers {

    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    @Column(name="ans_id")
    private int id;

    private String answers;

    @ManyToMany(mappedBy="answers")
    private List<Question> question;
}

```

Explanation

This class stores all Questions that use the same Answer.

Here,

```
private List<Question> question;
```

means one Answer can belong to multiple Questions.

Since this side uses mappedBy, it is called the Inverse Side.

Main Program

```

public class MultipleChoice {

    public static void main(String[] args) {

        Question question1 = new Question();
        question1.setQuestions("What is Hibernate");

        Question question2 = new Question();
        question2.setQuestions("What is JPA");

        Answers answer1 = new Answers();
        answer1.setAnswers("Hibernate is ORM framework!");
    }
}

```

```

        Answers answers2 = new Answers();
        answers2.setAnswers("JPA provide a set of rules which is followed by Hibe
rnative");

        Answers answer3 = new Answers();
        answer3.setAnswers("Hibernate is Framework which deal with database");

        List<Answers> ansList1 = new ArrayList<>();
        ansList1.add(answer1);
        ansList1.add(answers2);
        ansList1.add(answer3);

        question1.setAnswers(ansList1);

        List<Answers> ansList2 = new ArrayList<>();
        ansList2.add(answers2);

        question2.setAnswers(ansList2);

        Session currentSession =
            ConnectionEstablisher.getConnection().openSession();

        Transaction transaction =
            currentSession.beginTransaction();

        currentSession.persist(question1);
        currentSession.persist(question2);
        currentSession.persist(answer1);
        currentSession.persist(answers2);
        currentSession.persist(answer3);

        transaction.commit();
        currentSession.close();
    }
}

```

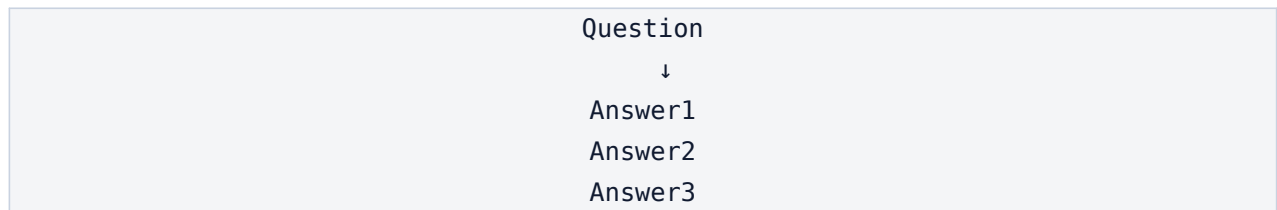
Understanding @ManyToMany

@ManyToMany

This annotation creates a relationship where many records from one table connect with many records from another table.

Why do we use List<Answers>?

One Question has multiple Answers.



So we use a List.

Similarly,

```
private List<Question> question;
```

allows one Answer to belong to multiple Questions.

Where do we write @ManyToMany?

Write it in both entities.

Owner Side:

```
@ManyToMany
private List<Answers> answers;
```

Inverse Side:

```
@ManyToMany(mappedBy="answers")
private List<Question> question;
```

Both classes know about each other.

When do we use it?

Use it when:

- One object has many related objects.
- Those objects also belong to many other objects.

Examples:

- Student ↔ Course
- Actor ↔ Movie

- User ↔ Role
- Question ↔ Answer

What happens if we don't write @ManyToMany?

Hibernate treats the List as a normal Java collection.

Problems:

- No relationship is created.
- No join table is created.
- Objects remain disconnected.

Understanding @ManyToMany(mappedBy="answers")

```
@ManyToMany(mappedBy="answers")
```

This tells Hibernate:

“The relationship is already managed by the answers field inside the Question class.”

Here,

```
private List<Answers> answers;
```

is the owner field.

So we write

```
mappedBy="answers"
```

because answers is the variable name.

What happens if we don't write mappedBy?

Both entities become owners.

Problems:

- Two join tables may be created.
- Duplicate mappings occur.
- Database structure becomes incorrect.

mappedBy keeps only one owner.

Understanding @ManyToMany(cascade = CascadeType.ALL)

```
@ManyToMany(cascade = CascadeType.ALL)
```

CascadeType.ALL tells Hibernate:

“Whatever operation happens on Question should also happen on its related Answers.”

If you perform

```
persist(question);
```

Hibernate can also save the related Answer objects.

Where do we write it?

Write it on the owner side.

Example:

```
@ManyToMany(cascade=CascadeType.ALL)
private List<Answers> answers;
```

When do we use Cascade?

Use it when the parent should automatically perform operations on related objects.

Operations included:

- Persist
- Merge
- Remove
- Refresh
- Detach

What happens if we don't use Cascade?

Then every related object must be saved manually.

Example:

```
persist(question);

persist(answer1);

persist(answer2);
```

Otherwise,

Hibernate may throw

```
TransientObjectException
```

because child objects are not saved.

Understanding @JoinTable

```
@JoinTable(  
    name="question_answer",  
    joinColumns=@JoinColumn(name="que_id"),  
    inverseJoinColumns=@JoinColumn(name="ans_id")  
)
```

This annotation creates the middle table that connects both entities.

Think of it as a bridge.

Parts of @JoinTable

1. name="question_answer"

Creates the bridge table.

2. joinColumns

```
joinColumns=@JoinColumn(name="que_id")
```

Stores the Question ID.

3. inverseJoinColumns

```
inverseJoinColumns=@JoinColumn(name="ans_id")
```

Stores the Answer ID.

Together they connect both tables.

What happens if we don't write @JoinTable?

Hibernate still creates a bridge table.

But names become auto-generated.

Example:

```
Question_Answers  
  
Question_que_id  
  
Answers_ans_id
```

Custom names make the database easier to understand.

Data Inserted

Question	Connected Answers
What is Hibernate	Hibernate is ORM framework!, JPA provide a set of rules..., Hibernate is Framework which deal with database
What is JPA	JPA provide a set of rules...

Notice that the second answer is shared by both questions.

Database Tables Created

Table: que_value

QUE_ID	QUESTIONS
1	What is Hibernate
2	What is JPA

Table: ans_value

ANS_ID	ANSWERS
1	Hibernate is ORM framework!
2	JPA provide a set of rules which is followed by Hibernate
3	Hibernate is Framework which deal with database

Join Table: question_answer

This table is automatically created by @JoinTable.

QUE_ID	ANS_ID
1	1
1	2
1	3
2	2

This bridge table stores all relationships.

Notice:

Question 1 is connected with Answers 1, 2, and 3.

Question 2 is connected with Answer 2.

Relationship Flow Inside the Database

The join table stores only IDs and connects both entities.

Execution Flow (Step by Step)

This is how the program runs internally.

Step 1: Create two Question objects.

```
Question 1
```

```
Question 2
```

Step 2: Create three Answer objects.

```
Answer1
```

```
Answer2
```

```
Answer3
```

Step 3: Create Answer lists.

```
List<Answers> ansList1;
```

Question 1 receives three answers.

Question 2 receives one shared answer.

Step 4: Connect both sides.

```
question1.setAnswers(ansList1);
```

and

```
answer1.setQuestion(queList1);
```

Now both entities know each other.

Step 5: Open Hibernate Session.

```
openSession();
```

Database connection starts.

Step 6: Begin Transaction.

```
beginTransaction();
```

Hibernate starts saving data.

Step 7: Save Questions.

```
persist(question1);  
  
persist(question2);
```

Parent records are inserted.

Step 8: Save Answers.

```
persist(answer1);  
  
persist(answer2);  
  
persist(answer3);
```

Child records are inserted.

Step 9: Hibernate creates the Join Table.

```
question_answer
```

Hibernate automatically inserts

```
(1,1)  
  
(1,2)  
  
(1,3)  
  
(2,2)
```

into the bridge table.

Step 10: Commit Transaction.

```
transaction.commit();
```

All records become permanent.

Step 11: Close Session.

```
currentSession.close();
```

Database resources are released.

Quick Revision

Annotation	Purpose
@ManyToMany	Creates a many-to-many relationship.
mappedBy	Makes the current class the inverse side.
CascadeType.ALL	Applies all operations to related entities.
@JoinTable	Creates the bridge table.
joinColumns	Stores the owner's foreign key.
inverseJoinColumns	Stores the inverse entity's foreign key.

Remember

- Question is the Owner Side.
- Answers is the Inverse Side.
- @JoinTable creates the bridge table.
- CascadeType.ALL performs operations on related entities.
- mappedBy prevents duplicate relationship mappings.
- The question_answer table stores only IDs and connects both entities.